FlashANNS: Stall-Aware Graph Search on Flash-Backed CXL Memory

Anonymous Author(s)

Abstract

Billion-scale approximate nearest neighbor search (ANNS) must scale corpus capacity and query throughput independently. Replicated block-SSD systems such as DiskANN pay a storage-replication tax as query hosts grow; full-in-memory CXL-DRAM systems such as CXL-ANNS and Cosmos pay a DRAM-capacity tax on the cold tail. Flash-backed CXL memory (CXL-SSD) is the third deployment point: a flash-priced corpus with a memory-semantic hot window and, under Shared FAM, a path to one-copy serving. It does not make NAND as fast as DRAM. Naive load/store search on this tier hits a two-order miss cliff, admits the wrong objects through page caches, pollutes the small DRAM window with low-precision prefetch, and collapses device parallelism to a blocking queue depth of one.

We present FlashANNS, a graph-ANNS runtime that jointly manages residency and stalls on flash-backed CXL memory. FlashANNS places the graph and PQ codes in host DRAM, promotes only high-precision frontier and hub records into a bounded window, shares that hot set across queries, controls search width under an iso-recall objective, and issues asynchronous memory-semantic fills with continuous fetch-level batching. On [**datasets**], at iso-recall@[k], FlashANNS achieves [**X**×] the throughput of naive CXL-SSD and comes within [**Y**] of a same-machine DiskANN/PipeANN block path, without lifting the corpus into CXL-DRAM. We evaluate a software CXL-SSD proxy, do not claim a measured 160 GB/s device-internal path, and treat multi-host one-copy as a conditioned deployment model rather than a result.

CCS Concepts: • **Computer systems organization** → **Secondary storage organization**; • **Information systems** → **Nearest-neighbor search**; • **Software and its engineering** → *Memory management*.

Keywords: CXL, CXL-SSD, approximate nearest neighbor search, graph ANNS, memory-semantic flash, residency, prefetch, continuous batching

1 Introduction

Billion-scale approximate nearest neighbor search (ANNS) must scale two *independent* dimensions: *corpus capacity* and *query throughput*. Adding query hosts does not, by itself, shrink the full-precision graph and embedding store; growing that store does not, by itself, create more CPU and I/O. Production serving therefore faces a deployment question that neither "use a better graph" nor "buy a faster SSD" answers:

> *When query load grows, can we add compute without replicating the cold corpus, and without lifting that corpus into DRAM-priced memory?*

Two published deployment points dominate the literature, and each taxes one of those dimensions. Block-SSD graph systems such as DiskANN [18, 19] and its pipelined descendants [9] are excellent at hiding NAND latency behind asynchronous block I/O, product quantization (PQ) navigation, and a per-host node cache. Their common serving unit, however, is still *CPU + DRAM working set + a local SSD index*. Scaling throughput by R query workers therefore typically replicates provisioned flash ($\propto R \cdot D$) or shards the graph and pays cross-shard fan-out. Remote block access (NVMe-oF) moves the cable, not the tax: each host still maintains its own page/buffer cache. Distributed serving [6] shows that a single logical DiskANN graph can live behind a shared KV store, but the interface remains block/KV, not a unified load/store address space.

The complementary point is full-in-memory CXL-DRAM ANNS. CXL-ANNS [14, 15] and Cosmos [4] put the graph and embeddings in a CXL memory pool so that search never stalls on flash. That design eliminates NAND misses, and a pooled HDM *can* be one copy — but it prices the *cold tail* at DRAM. For a corpus of size D whose hot working set H satisfies $H \ll D$, the stranded-capacity problem is structural [1, 2]: most vectors are rarely touched and still occupy a DRAM-class device.

Flash-backed CXL memory (CXL-SSD) is the missing third point (Figure 1). A Type-3 memory-semantic SSD exposes NAND capacity and a small device/host DRAM window through `load/store` on a CXL.mem address space [3, 7, 10, 13]. Relative to replicated DiskANN, the opportunity is to stop copying D with every added query host; relative to CXL-ANNS/Cosmos, it is to price the cold corpus at flash. Under CXL 3.x Shared FAM/GFAM, multiple hosts can map the same region concurrently [13] — a *deployment* path to one-copy serving, not a mechanism we claim to have evaluated. We state the three CXL conditions explicitly so they are not conflated: (i) direct-attach CXL-SSD already provides flash-priced memory semantics on one host; (ii) CXL

2.0 pooling shares a device, not a concurrently mapped HDM region; (iii) only Shared FAM is a standard premise for multi-host one-copy.

CXL-SSD does *not* make NAND as fast as DRAM. A cold vector load on this tier costs tens of microseconds — two orders of magnitude above a CXL-DRAM or host-DRAM hit (Figure 3). Treating the device as slightly slow memory, or as "mmap DiskANN", fails in four systematic ways that we measure in §3: (1) the miss cliff dominates end-to-end search once any material fraction of loads miss; (2) OS-style page admission is a poor match for high-dimensional graph adjacency and inflates NAND reads; (3) synchronous or high-coverage graph prefetch pollutes the small DRAM window and can *reduce* QPS; (4) a blocking load/store miss collapses device queue depth to ≈ 1, so a scheduler designed for `io_uring` depth can become a pessimization until fills are made asynchronous. Widening search (`efSearch` / beam) to "buy recall" further cools the working set and couples quality to stall.

This paper presents FlashANNS, a graph-ANNS *runtime* for flash-backed CXL memory. FlashANNS does not replace DiskANN's block-SSD path and does not assume that the corpus fits in CXL-DRAM. It answers the management problem that the new tier leaves open: *which* objects occupy the small memory-semantic window, *which* misses are issued, and *how* those misses stay overlapped with compute and with other queries under an iso-recall objective. The runtime has five coupled modules (§4): semantic placement; bounded precise promotion; a cross-query shared hot set; stall-aware search control; and continuous fetch-level batching over asynchronous memory-semantic reads.

To our knowledge, this is the first *graph-ANNS serving system* that directly operates over an SSD-backed CXL address space and jointly manages ANNS-specific residency, promotion, and search stalls. We do *not* claim to be the first work to attach a CXL-SSD to a vector database, nor that we have deployed Shared FAM in production.

This paper makes four contributions:

1. **A third deployment point, stated with its limits.** We separate the CXL-SSD *opportunity* (flash-priced capacity, optional one-copy) from the *system* (residency/stall control), and we name the CXL 2.0 vs. 3.x conditions under which one-copy is even well-formed (§1, §2).
2. **A measured pathology of graph ANNS on this tier.** We show that naive page caches, blind prefetch, unconstrained search width, and blocking load/store each amplify stall, and we turn the negative results into design constraints (§3).
3. **A complete ANNS-aware runtime.** FlashANNS jointly implements placement, bounded promotion, a shared hot set, iso-recall search control, and continuous miss batching (§4, §5).
4. **An iso-recall evaluation against the two published points.** At matched recall, we compare FlashANNS to naive CXL-SSD policies, to a same-machine DiskANN/PipeANN block path, and (as an upper bound) to an all-in-CXL-DRAM oracle, and we ablate each module (§6).

Scope we do not claim. The experimental device is a software CXL-SSD that services `load`/`store` misses from NAND (plus an optional commercial CXL-DRAM window). We do not report a measured 160 GB/s device-side DRAM–SSD path. Multi-host one-copy is a deployment narrative conditioned on Shared FAM; the prototype is single-host. Cost claims are qualitative provisioning curves, not TCO.

The rest of the paper is organized as follows. §2 reviews graph ANNS and CXL memory-semantic flash. §3 presents the findings that constrain the design. §4–§5 describe FlashANNS. §6 evaluates it. §7 and §8 place the work and state limits.

2 Background and Deployment Points

2.1 Graph ANNS and the block-SSD path

A graph ANNS index (Vamana [19], HNSW [16]) stores, for each vector, a small neighbor list. Search maintains a candidate set of width L (beam / `efSearch`), repeatedly expanding the closest unread node, and returns the top-k. Quality is monotone in how many nodes are scored; *stall* is monotone in how many of those scores miss the fast tier.

DiskANN [18, 19] is the reference block-SSD design. Full- precision vectors and adjacency live in 4 KB-aligned sectors on NVMe. A compact PQ codebook in DRAM scores neighbors without a NAND read; an `aio` beam issues several sector reads at once; a static node cache pins entry vertices. PipeANN [9] keeps that layout and overlaps compute with I/O inside a single query, and can interleave queries to raise device queue depth. The first-class objects of this path are *the I/O queue and the sector*. They are the wrong first-class objects once the device is no longer a block target but a faulting address space (§3).

2.2 CXL-DRAM ANNS

CXL.mem lets a host map device HDM into its physical address space [13]. CXL-ANNS [14, 15] places the essential graph and embeddings in a Type-3 CXL-DRAM pool

Fig. 1 placeholder — draw three columns:
(a) replicated local-SSD DiskANN ($R \times D$ flash, per-host cache);
(b) full-in-memory CXL-DRAM ANNS (D at DRAM price);
(c) FlashANNS: graph/PQ in host DRAM, hot records in a bounded CXL-DRAM window, full-precision corpus on CXL-SSD, optional Shared-FAM caption.

Figure 1. Three deployment points for billion-scale graph ANNS, and the FlashANNS runtime that sits on the third. CXL-SSD is an *opportunity* (flash-priced corpus, memory-semantic hot path, one-copy under Shared FAM). The *system* is residency and stall control on that tier. **Fill-in:** replace this box with a 3-column deployment drawing plus a compact runtime overlay on (c). Do not draw a 160 GB/s device-internal bus as a measured link.

and hides ∼100 ns far-memory delay with a relationship-aware cache and DSA-assisted prefetch. Cosmos [4] further offloads traversal and distance to device-side compute. Second-tier and residual-quantization designs [1, 11, 17] also assume a DRAM- or NVM-priced far tier. FlashANNS shares the *interface* (load/store) with this line and rejects its *capacity assumption* (the corpus fits in CXL-DRAM).

2.3 Flash-backed CXL memory

A CXL-SSD pairs NAND with a modest DRAM cache and presents a byte-addressable window [3, 7, 10]. A hit is a CXL.mem load; a miss is a NAND fill into that window, then a retry. The programming model looks like memory; the miss tax looks like flash. Hardware and generic OS work on this tier does not manage graph-ANNS residency. CXL-AnySSD [5] accelerates vector-DB traffic by removing swap, not by controlling promote precision or search stalls. P-HNSW [8] studies crash-consistent graphs and mentions CXL-SSD; its experiments use DRAM-simulated PM.

Three conditions must not be mixed when we talk about sharing (Figure 2):

1. **Direct-attach CXL-SSD.** One host, flash price, memory semantics. Sufficient for the single-host claims in this paper.
2. **CXL 2.0 pooling / MLD.** Several hosts may time-share a device; a given HDM region is owned by one host at a time.
3. **CXL 3.x Shared FAM / GFAM.** Several hosts may map the same region at once. This is the standard premise for multi-host one-copy serving, and additionally requires a fabric manager and a read-only publish protocol. We do not evaluate it.

Fig. 2 placeholder — CXL-SSD stack:
host CPU → CXL link → endpoint
(DRAM window + FTL + NAND).
Annotate: host-side link is per-host;
device-side bandwidth is shared.

Figure 2. Flash-backed CXL memory. Host-side CXL links are independent; device-side port, controller, DRAM cache, FTL, and NAND are shared. One-copy reduces replicated capacity; it does not replicate SSD IOPS. **Fill-in:** use the real endpoint you measure; mark the software proxy path if that is the prototype.

Media-cost sketch (qualitative). Let c_{DRAM} and c_{flash} be unit media prices and n_{HA} the number of high-availability copies ($n_{\text{HA}} \ll R$ in the common case). Then

$$\text{MediaCost}_{\text{CXL-DRAM}} \sim D \cdot c_{\text{DRAM}},$$

$$\text{MediaCost}_{\text{CXL-SSD}} \sim n_{\text{HA}} D \cdot c_{\text{flash}} + H \cdot c_{\text{cache}} + C_{\text{fabric}}.$$

We use this only to locate the third deployment point (Figure 11). We do not claim a lower TCO.

3 Why Naive CXL-SSD ANNS Fails

A unified VA does not make graph ANNS fast. This section records the pathologies that a runtime must treat as first-class. Each finding answers two questions: *does the effect exist in the numbers?* and *how much does being wrong cost?* We later require every design module to name the finding it answers (Table 1).

3.1 F1: A two-order miss cliff

A full-precision load that misses the CXL-SSD window costs [**CXL-SSD cold** μ**s**]; a hit in the software/device cache with a cold PTE costs [**soft** μ**s**]; a warm PTE hit costs [**warm** μ**s**] (Figure 3). The ratio of the first to the last is [∼**100–140**×]. Graph search multiplies this

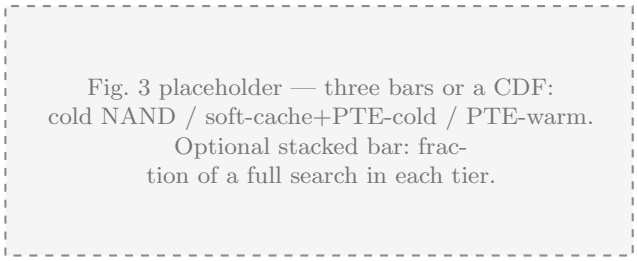

Figure 3. Miss cliff on flash-backed CXL memory. Soft-cache hits are *not* PTE hits; reporting only `mincore`/PTE occupancy hides the middle step. **Fill-in:** per-vector latency and a whole-search breakdown on the same index.

tax by hops × degree. Once a modest fraction of loads miss, wall-clock time is spent in the NAND path, not in distance arithmetic. DiskANN already knew this for block I/O; the new fact is that *memory-semantic* access does not remove the cliff — it only changes how a miss is issued (§3.5).

3.2 F2: Page admission mismatches the graph

Neighbor IDs are not sequential in the address space. Admitting 16 KB OS pages (or CXL-SSD cachelines grouped as pages) therefore brings vectors the search will not score. On a high-dimensional corpus where a vector is already close to a page, page admission versus per-record admission splits QPS by [∼**3.6**×] and NAND reads by [∼**3.7**×] (Figure 4a). The claim is scoped: at small dimensions several records share a page and the gap narrows. Default page-cache thinking is still the wrong default for this workload.

3.3 F3: Promotion volume without precision is harmful

Synchronous logical prefetch of 1-, 2-, or 8-hop neighborhoods leaves hit rate almost unchanged, multiplies NAND reads, and drops QPS by [∼**4**–**5**×] at the widest setting (Figure 4b). Precision (*used*/*promoted*) falls as coverage grows; leftover pages evict useful ones from a few-gigabyte window. *Wrong promotion costs more than a missed promotion.* The useful policy is a bounded, high-precision ensure of the nodes the search is about to expand — not "prefetch harder."

3.4 F4: Search width is a stall knob

Raising `efSearch` / beam grows the unique IDs touched per query, cools the window, and raises NAND reads faster than it raises recall (Figure 4c). Recall itself saturates once a hop budget is spent. A shared hot set at a *small* beam beats a cold cache at a *large* beam. A controller that chops beam on instantaneous miss rate does not, by itself, win at iso-recall: it has no recall constraint and no IO budget. Quality and stall must be controlled jointly.

Table 1. Findings become challenges become knobs. **Fill-in:** replace — with the headline measured delta (e.g., QPS ×, IO ×).

ID	Finding (exists + hurts)	Challenge	FlashANNS knob
F1	Miss cliff —×	C1 visibility of tiers	counters, cost model
F2	Page ≠ record —×	C2 placement	record / hub admission
F3	Low-precision promote —×	C3 bounded promote	top-M, byte budget
F4	Beam cools the set —	C5 iso-recall	beam / early stop
F5	Bubbles; mmap QD≈ 1	C4/C5 pipeline	async fill + `cont`
F7	Shared hot set > deep beam	C4 shared cache	cross-query pin

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

3.5 F5: A single query cannot fill the miss pipeline

Graph search is pointer-chasing. Even an excellent single-query pipeline leaves device-queue bubbles: the next hop is unknown until the current page is scored, the beam ramps slowly, and the query drains at the end (Figure 5). On NVMe, interleaving independent queries raises occupancy; a *continuous*, fetch-level scheduler that admits work at read granularity can hold a target depth D.

On CXL-SSD the same idea is necessary and insufficient. A blocking `memcpy` from a mapping is a synchronous NAND fault (effective QD≈ 1 per thread). A depth-oriented scheduler that issues *speculative* fills then pays their latency on the critical path and can lose to a narrower pipeline. Continuous batching pays off on this tier only after memory-semantic fills are asynchronous (Figure 8, §4.4).

3.6 From findings to design constraints

Table 1 is the contract for §4. We do not treat CXL-SSD as uniform memory (C1). We place the graph and PQ off the SSD window (C2). We promote few, precise records (C3) and share them across queries (C4). We control beam under iso-recall or an IO budget (C5), and we issue misses asynchronously at a target depth. Evaluation must compare against a block-SSD path on the same corpus and recall (C6).

Fig. 4 placeholder — three panels, same y-family (QPS or normalized):
(a) record vs. 16 KB page admission;
(b) demand vs. 1-/2-/8-hop sync prefetch (precision annotated);
(c) efSearch / beam sweep: QPS, hit%, recall@10, NAND reads.

Figure 4. Three default usages that amplify stall. (a) Page admission. (b) Blind graph prefetch. (c) Widening search. Each panel must keep recall visible so raw QPS cannot be misread. **Fill-in:** one protocol (dataset, L, cache bytes) across panels if possible.

Fig. 5 placeholder — in-flight depth vs. time:
(a) single-query pipeline; (b) static
batch; (c) continuous fetch batching.
(Existing TikZ in sec-
tions/fig_bubbles.tex can replace this box.)

Figure 5. Pipeline bubbles. y is in-flight device reads; D is the depth needed to exercise NAND parallelism. A single query cannot hold D; a static batch drains at the barrier; continuous fetch-level admission targets D at unchanged recall. **Fill-in:** swap in the measured schematic; add a CXL-SSD panel that shows blocking load/store stuck at QD≈ 1.

4 Design

FlashANNS is a userspace ANNS runtime plus a thin residency plane in front of host DRAM, an optional CXL-DRAM window, and a flash-backed CXL address space (Figure 6). Search is still best-first graph walk. What changes is *where* each object lives, *which* cold records are made resident, and *when* a miss is issued relative to other work.

4.1 D1: Semantic placement

Table 2 is the default map. The graph adjacency and the PQ codebook are touched on every hop and are small relative to the full-precision corpus; they live in host DRAM so they never compete with vectors for the CXL-SSD window (F2, F7). Query state (candidates, visited set, thread-local arenas) is likewise host-local. Full-precision vectors default to the CXL-SSD address space. A bounded *hub / shared hot set* of records is explicitly cached into the window or into CXL-DRAM (§4.3). Frontier-top neighbors are candidates for precise promotion, not for bulk 2-hop prefetch (F3).

Table 2. Default placement. **Fill-in:** bytes on the corpus you report (e.g., 25M × dim).

Object	Default tier	Why
Graph (R neighbors)	host DRAM	every hop; keep off the v
PQ / codebook	host DRAM	tiny, extremely hot
Query / visited	host DRAM	per-worker state
Full-precision vec.	CXL-SSD	capacity
Hub / shared hot set	window / CXL-DRAM	F7
Frontier-top nbrs	bounded promote	F3, F5

This is a deliberate difference from DiskANN, where adjacency and vectors share SSD sectors. We pay a modest host-DRAM tax for the graph in order to stop the window from turning into an accidental adjacency cache.

4.2 D2: Bounded precise promotion

Promotion is an explicit `promote(ids[], level, budget)` on the residency plane (Figure 7). The only ids that enter the call are the neighbors of the *current frontier top-M* — the nodes the walk will expand next. A global and a per-query byte budget cap how much may move; overflow falls back to demand fill. The target level is a choice (F1): install into the device/software cache only, or also install PTEs. The runtime tracks precision = used/promoted online and shrinks M when precision falls below a threshold.

We forbid the policies F3 already falsified: synchronous high-coverage 2-hop prefetch, and "fault harder" loops that raise queue depth by touching unused pages. Over-promotion is the failure mode the byte budget exists to prevent.

On corpora where several vectors share a 4 KB page (low dimension), a *page-bin / reorder* layout clusters IDs that the walk co-touches so that one fill admits several useful records. This is layout serving residency, not a new index family. We report it as a placement option

Figure 6. FlashANNS architecture. The residency plane is the only path that moves full-precision bytes between CXL-SSD and the hot window. Search never page-faults a neighbor "to be helpful." **Fill-in:** use the implemented component names; mark the software-proxy device explicitly.

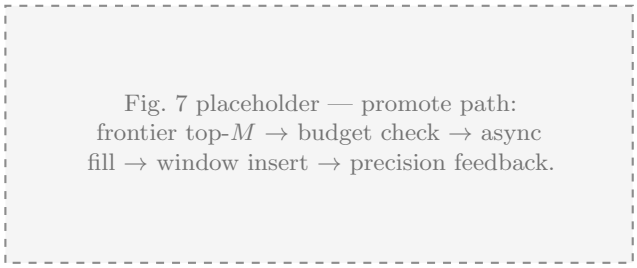

Figure 7. Bounded precise promotion. Only soon-to-expand neighbors are eligible; a byte budget and a live precision estimate keep the window from becoming a junkyard. **Fill-in:** annotate the implemented ioctl / batch path and the default M / budget used in §6.

and show when it helps (small dim) and when it does not (vector ≈ page).

4.3 D3: Shared hot set

Single-query deepening is a bad way to spend a small window (F4, F7). FlashANNS maintains a cross-query hot set: a fixed byte budget (a configured fraction of the CXL-SSD DRAM window) holding either high in-degree hubs or IDs that a short sliding window of queries repeats. Replacement is LFU with aging so a burst cannot freeze the set. The hot set is *decoupled* from beam: the default is a modest beam plus a warm shared set, not a large beam on a cold device.

The API is `cache(ids[])` / `uncache(ids[])`. Pinning is never implicit in a page fault. Multi-tenant isolation, if enabled, partitions the budget; the paper evaluation uses a single tenant unless stated.

4.4 D4: Stall-aware search and continuous batching

The search controller maximizes QPS at a recall@k floor (or maximizes recall at a fixed time/IO budget). It is *not* a one-shot "if miss rate is high, cut beam" rule. Offline, we record beam → (recall, IO) curves. Online, the controller picks the smallest beam that meets the recall floor, or stops when marginal recall per additional IO falls below a threshold. When D3 reports a high hot-set hit rate the controller may raise beam slightly, because the extra candidates are cheap.

Continuous fetch-level batching is the mechanism that makes that control loop feasible on a slow miss path (Figure 8). A global scheduler keeps a target number D of memory-semantic fills in flight, admitting the next read when one retires — not when a query, a hop, or a static group finishes. Fills are issued with an asynchronous batch interface so that D concurrent NAND operations actually reach the device. Scoring of already-resident records proceeds on the host and is never queued behind a fill it does not depend on.

4.5 D5: Observability

Every experiment in §6 is required to report the same counters the controller sees: ΔNAND reads, Δfaults, soft-cache / PTE / path hits, promote precision, hot-set hit rate, in-flight depth, and iso-recall@k. QPS without those counters is not a result. The residency plane exports `stats()` for this purpose.

What we refuse to hide. If a module cannot move the iso-recall curve in an ablation, it is removed from the contribution list rather than kept as decoration. That is the bar for §6.4.

5 Implementation

FlashANNS is implemented as a userspace runtime on top of a memory-semantic CXL-SSD device node and, when present, a CXL-DRAM DAX window. The prototype we evaluate uses a software CXL-SSD: `mmap` of the device presents a unified VA; a miss is a NAND fill into a configured DRAM cache; a batch ioctl submits many fills without one blocking bio per page. Graph search is integrated with a DiskANN/PipeANN-style engine so that the block-SSD baseline and the CXL-SSD

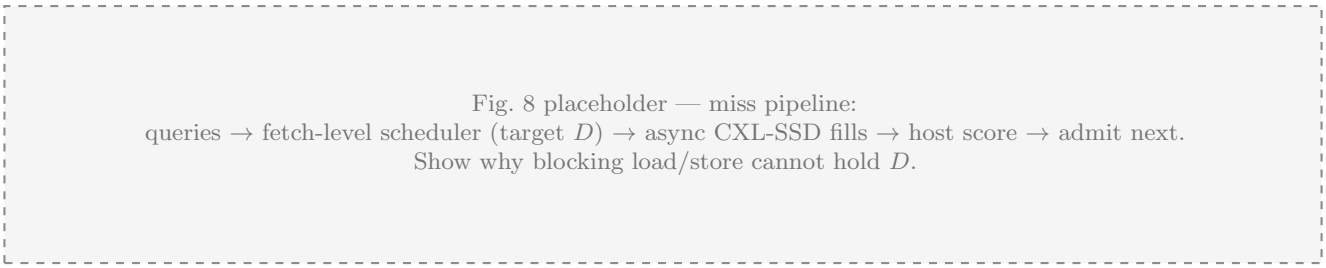

Figure 8. Continuous fetch-level batching over asynchronous memory-semantic fills. Recall is unchanged by construction: prefetch only changes *when* a needed page is resident, not *which* nodes the walk visits. **Fill-in:** draw the implemented `cont` path; put the blocking-mmap failure mode on the same figure.

path share the same index bytes and the same recall definition.

What is implemented. (1) Placement of graph and PQ in host DRAM and vectors on the CXL-SSD mapping (§4.1). (2) The promote/cache/stats residency API, including a byte budget and precision counters (§4.2, §4.5). (3) A shared hot-set with a fixed budget (§4.3). (4) Asynchronous frontier prefetch and a fetch-level continuous scheduler (§4.4). (5) An iso-recall harness that sweeps beam / L and stops at a target recall@k.

What is not implemented, and is not claimed. A hardware CXL 3.x Shared FAM fabric; a device-side 160 GB/s DRAM–SSD DMA engine; a production multi-tenant QoS layer; and a crash-consistent index publisher. The iso-recall *controller* is implemented as a calibrated policy plus early-stop; a fully closed online learner is optional and is reported only if it appears in the ablation. Turning off PQ navigation on the official DiskANN SSD path required an experimental flag; we never present that flag as a supported upstream feature.

Prototype scale. The runtime is [**LOC**] lines of C/C++ beyond the unmodified search engine, plus [**LOC-driver**] lines in the software CXL-SSD path. Build and identity checks are scripted so that a device other than the one we measured cannot silently produce a "CXL-SSD" number.

6 Evaluation

We answer five questions:

- **Q1** Does naive CXL-SSD graph ANNS fail in the ways §3 claimed?
- **Q2** At iso-recall, does the full FlashANNS stack approach a same-machine DiskANN/PipeANN block path without putting the corpus in CXL-DRAM?
- **Q3** Does each module (placement, promote, hot set, controller, async/`cont`) move the iso-recall curve?
- **Q4** Do the trends hold as n, dimension, and concurrency change?

Table 3. Platform and corpora. **Fill-in:** lock one primary corpus for Q2 and put the rest in sensitivity.

Item	Configuration
Host CPU / DRAM	—
CXL-DRAM (optional)	— (or "absent this boot")
CXL-SSD	software proxy → NVMe —
Window / cache	— GiB DRAM window, — GiB device cache
Block-SSD baseline	same NVMe, `O_DIRECT/io_uring`
Primary corpus	— (dim, metric, n)
Sensitivity	—
Index	Vamana/HNSW, R =—, PQ = —
Target quality	recall@10 and recall@100 floors —
Protocol	cold: reset window; warm: stated separately

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

- **Q5** Where does the system lose, and is the loss honest?

6.1 Setup

Table 3 is the locked configuration. The primary metric is *iso-recall QPS* (and p99) at a stated recall@k floor. We always report NAND reads, promote bytes, and the three-level hit breakdown. A run that does not declare cold vs. warm is discarded: the window is small enough that accidental warm-up invents speedups.

Baselines. (1) **Demand:** CXL-SSD load/store, no promote. (2) **Page:** 16 KB admission. (3) **Sync-2hop:** the F3 policy. (4) **Static-wide:** large beam, no controller. (5) **Miss-chop:** cut beam on instantaneous miss (the F9 policy). (6) **DiskANN/PipeANN:** official block path, same machine, same corpus, same recall floor; PQ on unless a figure is labeled `--no_pq_nav`. (7) **Oracle:** corpus already in CXL-DRAM / host DRAM (upper bound).

We never present a home-grown `O_DIRECT` reader as "DiskANN."

6.2 Q1: Naive CXL-SSD fails

Figure 3–5 are reproduced on the evaluation machine under the Table 3 protocol so that Design and Evaluation share one dataset. We expect the same qualitative shape: a two-order cliff, page admission and blind prefetch losing to demand, beam cooling the set, and blocking load/store stuck near QD≈ 1. If a finding does not reproduce, it is dropped from the intro, not explained away in a footnote.

6.3 Q2: End-to-end iso-recall (the money plot)

Figure 9 and Table 4 are the submission's main result. The claim we will defend is: at the locked recall floor, FlashANNS delivers [**X**×] Demand and comes within [**Y**] of the block-SSD path, while keeping the corpus on flash and host DRAM at [**graph+PQ**] rather than D. If absolute QPS loses to DiskANN, we still report it and shift emphasis to IO efficiency and DRAM footprint — we do not hide the bar.

6.4 Q3: Each module is necessary

Figure 10 and Table 5 are the falsifiability test for contribution 3. We also sweep promote M and hot-set fraction to show the F3 lesson in a positive form: there is a budget at which precision and QPS peak, and past it they fall.

6.5 Q4: Scale and the one-copy opportunity

Figure 11a sweeps corpus size, dimension, and concurrency. Figure 11b is the only place we plot the one-copy opportunity: provisioned flash/DRAM as a function of R, using the media-cost sketch of §2. The measured prototype is still one host; the curve is a model, checked against the observation that device-side bandwidth is shared (Figure 2).

6.6 Q5: Losses, overhead, and fairness

We report the cases FlashANNS loses or looks worse: warm working sets that already fit the window (async prefetch ≈ demand); low dimension where page≈record; the residual gap to PQ-navigated DiskANN; and software-proxy artifacts (global cache lock, QD ceiling). Host-DRAM consumed by the graph and PQ is a cost, not a footnote. CPU is reported because DiskANN on this class of machine is I/O-bound, not CPU-bound; if FlashANNS becomes CPU-bound we say so.

Takeaway. Q1–Q4 must be answerable from the figures in this section alone. If the money plot needs a verbal caveat that changes the claim, the claim in §1 is the thing to change.

7 Related Work

Table 6 is the comparison we want a reviewer to use. Prose below only records the one-line difference; it does not repeat the background.

Block-SSD and distributed graph ANNS.. DiskANN, FreshDiskANN, PipeANN, and SPANN [9, 12, 18, 19] optimize sector layout, PQ navigation, and I/O overlap on a block device. DistributedANN [6] serves one logical graph from a KV store. FlashANNS is not "a better `io_uring` scheduler." Its first-class objects are residency, promotion precision, and memory-semantic miss depth. NVMe-oF and KV sharing still present a block/KV slow path and a per-host cache.

CXL-DRAM and near-memory ANNS.. CXL-ANNS and Cosmos [4, 14, 15] — and second-tier / residual designs [1, 11, 17] — assume a DRAM- or NVM-priced far tier and optimize for ∼100 ns, not for tens of microseconds of NAND plus a small polluted window. Their caches and prefetchers are not transferable at that delay gap (F3). We do not require Type-2 / PNM compute.

CXL-SSD hardware and generic software. SkyByte, From-Block-to-Byte, CMM-H, and CXL-AnySSD [3, 5, 7, 10] build or evaluate the medium. They do not jointly manage graph-ANNS residency, bounded promotion, and stall-aware search. P-HNSW [8] is about crash consistency on PM, not serving residency on CXL-SSD.

8 Discussion and Limitations

Proxy versus product. The measured CXL-SSD is a software device that fills NAND behind `mmap`. Latency cliffs, QD collapse, and promote pollution are properties of *memory-semantic flash*, not of one driver. Absolute QPS and lock artifacts may move on a hardware BAR path; we will not relabel the proxy as a shipping CXL.mem SSD.

One-copy is a deployment claim, not a result. Figure 11b is a provisioning model. Concurrent multi-host mapping needs CXL 3.x Shared FAM, a fabric manager, and a read-only publish protocol [13]. CXL 2.0 pooling is not that. Even with one copy, device-side IOPS remain shared: one-copy removes capacity replication, not bandwidth replication.

DiskANN remains the block-path champion. PQ navigation plus deep `aio` is extremely good at not reading full-precision vectors. FlashANNS is the runtime for the tier where those vectors *do* live in a load/store flash address space. A residual QPS gap on the PQ-on block path is an expected con, not a surprise we omit.

Layout tax is dimension-dependent. Record-vs-page (F2) is strongest when a vector is comparable to a page. Low-dimension corpora need the page-bin experiment; we will not universalize a 1024-d result.

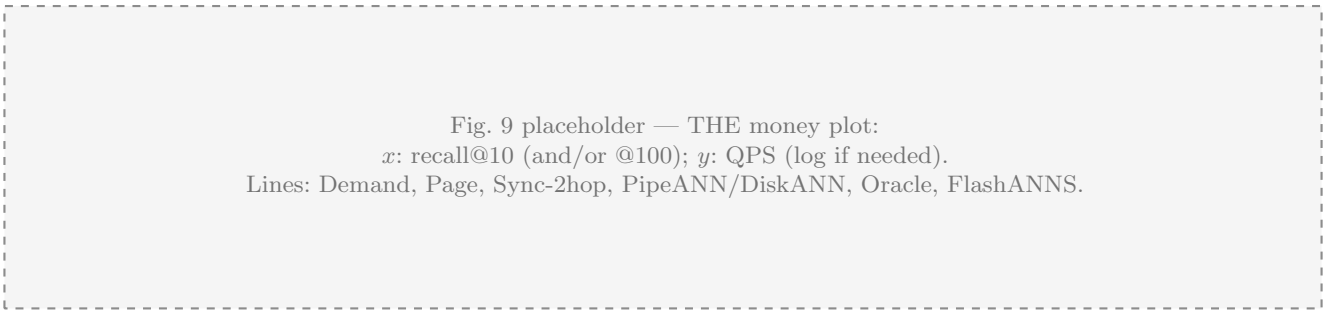

Figure 9. Iso-recall throughput. A 4.5 paper is won or lost on this figure. FlashANNS must beat every naive CXL-SSD policy by a clear gap and must state the residual gap to DiskANN/PipeANN and to the oracle — including when we lose. **Fill-in:** one primary corpus; cold protocol; error bars or repeated runs if variance is visible.

Table 4. End-to-end at the primary recall floor. **Fill-in:** one row per baseline; do not omit IO or DRAM just because QPS looks good.

System	Recall@10	QPS	p99 (μs)	NAND reads/q	Promote B/q	Host DRAM
Demand CXL-SSD	—	—	—	—	—	—
Page admission	—	—	—	—	—	—
Sync 2-hop	—	—	—	—	—	—
Static wide beam	—	—	—	—	—	—
Miss-chop beam	—	—	—	—	—	—
DiskANN (PQ on)	—	—	—	—	—	—
PipeANN block	—	—	—	—	—	—
Oracle CXL-DRAM	—	—	—	0	—	—
FlashANNS	—	—	—	—	—	—

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

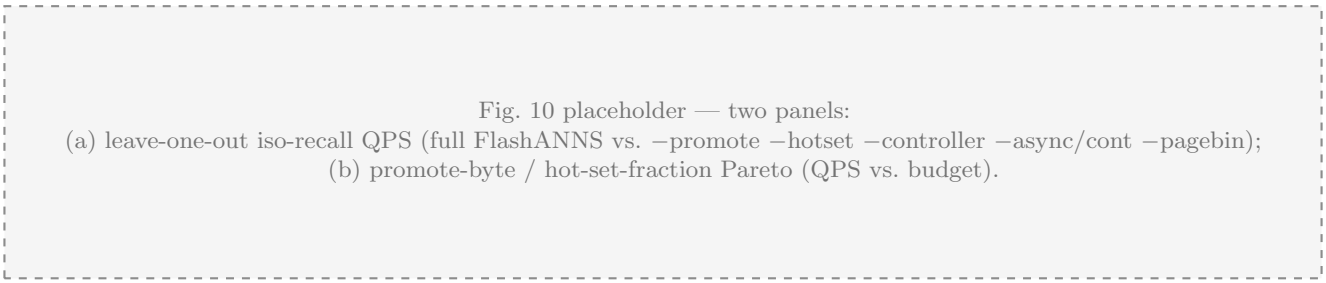

Figure 10. Ablation and budget Pareto. Removing any contributed module must move the main metric. The Pareto must show a bounded sweet spot, not "more cache is always better." **Fill-in:** same recall floor as Figure 9.

What would change our mind. If a hardware CXL-SSD with device-side copy saturates a wide internal bus, the *async issue* path stays, the *depth* target changes. If Shared FAM is available, D3 becomes a multi-host coherence/publish problem we have not solved. If bounded promote never beats demand on the primary corpus, contribution 3 shrinks to shared hot set + stall-aware scheduling and we will say so.

9 Conclusion

Graph ANNS has a well-engineered block-SSD path and a well-engineered CXL-DRAM path. Flash-backed CXL memory is the third deployment point — flash-priced capacity, a memory-semantic window, and a future one-copy hosting story — but it is unusable if treated as slow DRAM or as mmap'd DiskANN. FlashANNS is a runtime for that tier: it places the graph off the window, promotes few precise records, shares a hot set, controls search under iso-recall, and keeps NAND fills asynchronous and deep. The opportunity is the medium;

Table 5. Leave-one-out at the primary recall floor.

Variant	QPS	NAND/q	Hit (win/PTE)
Full FlashANNS	—	—	—
− bounded promote	—	—	—
− shared hot set	—	—	—
− iso-recall control	—	—	—
− async + `cont`	—	—	—
− page-bin (if used)	—	—	—
Demand only	—	—	—

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

Fig. 11 placeholder — two panels:
(a) QPS vs. n / dim / threads at iso-recall;
(b) provisioned media vs. R query hosts (replicated SSD vs. CXL-DRAM vs. CXL-SSD n_{HA}).

Figure 11. Scaling and provisioning. (a) Trends must not collapse on the primary knob range. (b) Qualitative capacity tax as query hosts grow; this is the Route A opportunity figure, not a TCO claim, and not a Shared-FAM measurement. **Fill-in:** (b) can be analytic plus one measured single-host point; label it as a model.

the contribution is residency and stall control. We have stated where the prototype is a proxy and where the paper is a model, so that those sentences cannot migrate into the abstract as facts.

References

[1] Anonymous. 2024. Second-Tier Memory for Vector Search. *arXiv preprint* (2024). Replace with the Second-Tier Memory citation..

[2] Anonymous. 2025. Bauhaus: CXL Memory Tiering and Stranded Capacity. *Placeholder* (2025).

[3] Anonymous. 2025. CMM-H: CXL Memory Module with NAND Backing. *Placeholder* (2025). Replace with the CMM-H citation..

[4] Anonymous. 2025. Cosmos: CXL-Based Full In-Memory Approximate Nearest Neighbor Search with Near-Data Processing. *IEEE Computer Architecture Letters* (2025). Replace with the camera-ready Cosmos / CAL'25 citation..

[5] Anonymous. 2025. CXL-AnySSD: Composable CXL SSDs without Swapping. (2025). UIUC thesis; replace with the public citation..

[6] Anonymous. 2025. DistributedANN: Serving a Logical DiskANN Graph over a Distributed Key-Value Store. *arXiv preprint arXiv:2509.06046* (2025).

[7] Anonymous. 2025. From Block to Byte: Rethinking SSDs with CXL. In *Placeholder*. Replace with the From-Block-to-Byte citation..

[8] Anonymous. 2025. P-HNSW: Crash-Consistent Hierarchical Navigable Small World Graphs on Persistent Memory. *Placeholder* (2025).

[9] Anonymous. 2025. PipeANN: Pipelined Graph-Based Approximate Nearest Neighbor Search on SSD. In *USENIX Symposium on Operating Systems Design and Implementation*. Replace with the camera-ready PipeANN / OSDI'25 citation..

[10] Anonymous. 2025. SkyByte: Architecting an Efficient Memory-Semantic CXL-based SSD. In *Placeholder*. Replace with the SkyByte citation..

[11] Anonymous. 2026. FaTRQ: Far-Memory-Aware Residual Quantization for Vector Search. *arXiv preprint* (2026). Replace with the camera-ready FaTRQ citation..

[12] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search. In *Advances in Neural Information Processing Systems*.

[13] CXL Consortium 2023. *Compute Express Link Specification, Revision 3.1*. CXL Consortium.

[14] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *USENIX Annual Technical Conference*.

[15] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2024. Bridging Software-Hardware for Billion-Scale ANNS with CXL Memory Disaggregation. *ACM Transactions on Computer Systems* (2024).

[16] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

[17] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Advances in Neural Information Processing Systems*.

[18] Aditi Singh, Suhas Jayaram Subramanya, Rohan Kadekodi, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. In *arXiv preprint arXiv:2105.09613*.

[19] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*.

A Fill-in Checklist

Table 7 is the author-facing map from each placeholder to the result file that should replace it. Delete this appendix before a submission if it is still author notes; or keep a cleaned artifact appendix after the numbers exist.

B Artifact Outline

An ASPLOS artifact should include: the identity gate for the CXL-SSD node; scripts that reset the window (cold protocol); the iso-recall harness; and the exact command lines that produce Figures 9–11. We will not point a reviewer at an identifiable internal hostname.

Table 6. Related systems on the dimensions that actually differ. **Fill-in:** keep these columns; add a row only if it changes a column.

System	Full-prec. home	Graph home	Slow path	Capacity assumption	Optimize for	Cache unit
DiskANN / PipeANN	NVMe sectors	with vectors	async block I/O	flash is enough	IOPS, PQ trips	4 KB / entry cache
Second-Tier / FaTRQ	CXL/RDMA/NVM	varies	far load	second tier holds index	4 KB vs. fine read	fine-grained far
CXL-ANNS	**all CXL-DRAM**	CXL	CXL.mem + DSA	pool holds corpus	∼100 ns + near-data	relation cache
Cosmos / PNM	device DRAM	device	on-device cores	multi-device DRAM	offload walk/dist	device place
CXL-SSD HW / AnySSD	NAND HDM	n/a	fault / generic	flash + window	composable SSD, no swap	pages / OS
FlashANNS	**CXL-SSD**	**host DRAM**	fault + async fill	flash required; not all-DRAM	residency + stall	record / hub / top-M

Table 7. Placeholder → intended source. Author notes; not a scientific result.

Id	Type	Replace with
Fig. 1	figure*	3-column deployment + runtime overlay
Fig. 2	figure	Measured/proxy CXL-SSD stack
Fig. 3	figure	Cold / soft / warm latency + search breakdown
Fig. 4	figure*	Page vs. record; prefetch precision; beam sweep
Fig. 5	figure	`sections/fig_bubbles.tex` or a measured redraw
Tab. 1	table	Headline deltas from MOTIVATION_FINDINGS
Fig. 6	figure*	Implemented component names
Tab. 2	table	Byte counts on the primary corpus
Fig. 7	figure	Promote API + default M/budget
Fig. 8	figure*	`cont` + async fill; blocking mmap failure
Tab. 3	table	Locked platform and corpora
Fig. 9	figure*	Iso-recall QPS (money plot)
Tab. 4	table*	End-to-end row per baseline
Fig. 10	figure*	Leave-one-out + Pareto
Tab. 5	table	Leave-one-out numbers
Fig. 11	figure	Scale sweep + provisioning model
Tab. 6	table*	Camera-ready citations